

Sprint 1 Report

FinLit India — Gamified App for Financial Knowledge
Programming Project — THWS

Vadivelan Sangeetha Nisanth, Vel Dhayaa, Iurkov Vsevolod

May 2026

Contents

1	Project Overview	2
2	Sprint 1 Goal	2
3	System Architecture	2
3.1	Technology Stack	2
4	Backend Implementation	2
4.1	API Endpoints	2
4.1.1	Authentication	3
4.1.2	Lessons	3
4.1.3	Progress	3
4.1.4	Quiz	3
4.2	Data Model	3
5	Frontend Implementation	4
5.1	Application Flow	4
5.2	Screens	4
5.2.1	Registration Screen	4
5.2.2	Roadmap Screen	4
5.2.3	Lesson Screen	4
5.2.4	Profile Screen	5
6	Completed Functionality	5
7	Known Limitations	5
8	Summary	6

1 Project Overview

FinLit India is a mobile application designed to teach basic financial literacy to women in rural India. The application follows a gamified, incremental learning model inspired by Duolingo: users work through a visual lesson roadmap, read short articles, answer quizzes, and earn coins upon completion.

The target audience consists of women in rural Indian communities who may have limited prior experience with digital devices. The interface is intentionally kept simple, with large touch targets, minimal text, and a clear visual hierarchy.

The project is developed as part of the Programming Project course at Technische Hochschule Würzburg-Schweinfurt (THWS).

2 Sprint 1 Goal

The goal of Sprint 1 was to deliver a functional end-to-end prototype demonstrating the core user flow: registration, lesson discovery, lesson completion, and progress tracking. No production-readiness or deployment was required at this stage.

3 System Architecture

The prototype consists of two layers:

- **Frontend:** A Flutter application targeting Android (primary), with platform scaffolding also generated for iOS, Web, macOS, Linux, and Windows.
- **Backend:** A Python REST API built with FastAPI, connected to Google Firebase Firestore as the data store.

The Flutter application communicates with the backend exclusively via HTTP, using a dedicated `ApiService` class. The backend exposes a RESTful API and initialises a Firebase Admin SDK client to read and write Firestore documents.

3.1 Technology Stack

Component	Technology
Mobile framework	Flutter (Dart)
Backend framework	FastAPI (Python)
Database	Google Firebase Firestore
Local storage	SharedPreferences (on-device)
Version control	GitLab (university instance)

4 Backend Implementation

4.1 API Endpoints

The backend exposes ten REST endpoints grouped into four functional areas.

4.1.1 Authentication

- **POST /auth/register** — Accepts a phone number, checks the **users** Firestore collection for duplicates, and creates a new user document with **name=null**, **language=null**, and a server timestamp. Returns the generated **user_id**.
- **POST /auth/profile** — Updates the **name** and **language** fields of an existing user document. Returns 404 if the **user_id** is not found.

4.1.2 Lessons

- **GET /lessons** — Returns all lesson documents from the **lessons** Firestore collection as a flat JSON array, including fields for title, description, content, module number, section name, coin reward, and sort index.
- **GET /lessons/{lesson_id}** — Returns a single lesson document by its Firestore document ID.

4.1.3 Progress

- **POST /progress** — Writes or overwrites a progress document at **users/{user_id}/progress/{lesson_id}**, recording the completion flag and a server timestamp.
- **GET /progress/{user_id}** — Streams all documents from the **users/{user_id}/progress** sub-collection and returns them as a list.

4.1.4 Quiz

- **GET /quiz/{lesson_id}** — Reads the **questions** sub-collection of a lesson document. Returns each question with its four answer options and the correct option index.
- **POST /quiz/result** — Accepts a list of user answers, fetches the corresponding questions from Firestore, computes a percentage score, and writes the result to **users/{user_id}/quiz_results**.

4.2 Data Model

Firestore is used as a document database. The top-level collections and their relevant fields are summarised below.

Collection: users

Each document represents a registered user and contains: **phone** (string), **name** (string or null), **language** (string or null), and **created_at** (timestamp). Two sub-collections are attached to each user document:

- **progress** — One document per completed lesson, keyed by **lesson_id**, storing **is_completed** and **completed_at**.
- **quiz_results** — Auto-ID documents storing **lesson_id**, **score** (integer, 0–100), and **completed_at**.

Collection: lessons

Each document represents one lesson and contains: **title**, **description**, **content** (plain-text article), **moduleNumber** (integer), **sectionName** (string), **coinsReward** (integer), and **order_index** (integer). A **questions** sub-collection contains one document per quiz question, with fields **question**, **option_1** through **option_4**, and **correct_option** (integer, 1–4).

5 Frontend Implementation

5.1 Application Flow

On first launch, the application displays the registration screen. The phone number entered by the user is submitted to `POST /auth/register`; on success, the returned **user_id** is persisted to `SharedPreferences` and the user is navigated to the roadmap. On subsequent launches, the application detects the stored **user_id** and navigates directly to the roadmap, bypassing registration.

5.2 Screens

5.2.1 Registration Screen

A single-field form prefixed with the Indian country code (+91). On submission, the screen calls the registration endpoint, stores the returned **user_id** locally, and navigates to the roadmap.

5.2.2 Roadmap Screen

The main screen of the application. The top bar displays the application title alongside three gamification counters (streak, coins, gems). Below the top bar, a full-width green section banner identifies the current module. The lesson path is rendered as a scrollable sequence of circular nodes arranged in a zigzag pattern, connected by cubic Bézier curves drawn with a `CustomPainter`. Each circle has one of three visual states:

- **Completed** (green with a check mark): lesson has been finished.
- **Available** (green outline with lesson number): the next lesson to attempt.
- **Locked** (grey with a lock icon): requires prior lessons to be completed first.

Tapping an available or completed circle fetches the corresponding quiz questions from the API and opens the lesson screen. Tapping a locked circle shows an informational dialog. A bottom navigation bar provides access to the Home, Shop (placeholder), and Profile tabs.

5.2.3 Lesson Screen

Displays the lesson article followed by an inline quiz. Each question is presented as a card with four selectable options labelled A through D. After the user has answered all questions and tapped the submit button, the options animate to reveal correct (green) and incorrect (red) choices. A modal results dialog then presents the score as a percentage,

the number of coins earned, and a button to return to the roadmap. On submission, a progress record is written to the backend via `POST /progress`.

5.2.4 Profile Screen

Shows the user's phone number, a static learner badge, and three statistic cards (streak, coins, gems — currently displaying zero). A fourth card shows the total number of completed lessons, retrieved from `GET /progress/{user_id}`.

6 Completed Functionality

The following end-to-end flows were verified working by the end of Sprint 1:

- User registration via phone number, with duplicate detection and local session persistence.
- Returning users bypass registration on subsequent launches.
- Lessons are fetched from Firestore and displayed on the roadmap, grouped by module and section.
- Tapping an available lesson opens the article and quiz.
- Submitting quiz answers grades them locally, displays a results dialog, and records progress to Firestore.
- Completed lessons are visually marked; the immediately following lesson becomes available.
- The profile screen correctly reflects the number of completed lessons from Firestore.

7 Known Limitations

The following limitations were identified during Sprint 1 and are recorded for resolution in subsequent sprints.

1. **No phone verification:** Registration creates a Firestore user immediately without OTP or any other form of phone ownership verification.
2. **Gamification values not persisted:** Streak, coins, and gems are always displayed as zero. The backend and data model do not yet implement accumulation logic.
3. **Quiz results not submitted to backend:** The `POST /quiz/result` endpoint exists and functions correctly, but the frontend does not call it. Quiz answers are graded locally for display only.
4. **Lesson ordering not enforced:** Lessons have an `order_index` field, but neither the backend query nor the frontend sorts by it. Display order depends on Firestore insertion order.
5. **Profile screen detail list not rendered:** The count of completed lessons is displayed correctly, but the list of individual completed lesson cards does not render due to a hardcoded empty list in the current implementation.

8 Summary

Sprint 1 successfully delivered a functional end-to-end prototype of FinLit India. The core user journey — from registration through lesson discovery, content consumption, quiz completion, and progress persistence — operates correctly. Both the Flutter frontend and the FastAPI backend are integrated with Firebase Firestore, and the application runs on an Android emulator connected to a locally hosted backend instance.

The prototype provides a solid foundation for future sprints, which will address the known limitations, implement the remaining gamification features, and work toward a deployable release.